
Installation

Library Hackathon: AI Workflows Without Coding

Langflow 1.12.3 · September 2026

Gefördert durch:



The AI Service Centre Berlin Brandenburg is funded by the Federal Ministry of Research, Technology and Space under the funding code 16IS22092.

This guide explains how to set up the Library Hackathon kit on a laptop, from installing Docker to running the first flow. It is written for participants setting up their own laptop and for organisers preparing several laptops. No programming is involved.

Setting up takes about 20 to 30 minutes, most of it waiting for downloads. If the organisers have already prepared your laptop, go straight to step 5.

Contents

1. What you need
2. Install Docker
3. Download the hackathon files
4. Enter your settings
5. Start Langflow
6. Check that everything works
7. Optional: a local AI model
8. For organisers: preparing many laptops
9. Stop, update and remove
10. Troubleshooting

1. What you need

	Minimum	For a local AI model
Memory (RAM)	8 GB	16 GB
Free disk space	10 GB	15 GB
Graphics	not needed	ideally a Mac with an Apple chip, or a laptop with an NVIDIA graphics card
Operating system	Windows 10/11, macOS, or Linux	same

You also need:

- **Administrator rights** on the laptop, to install Docker.
- **Internet access.** Langflow is downloaded once (just under 1 GB), and the library components need to reach Crossref, OpenAlex, Unpaywall and Iobid while you work.
- **Credentials from the organisers** for the AI cluster: an address, a group key and a model name. Without them you can only use a local model (see step 7).

2. Install Docker

Langflow runs inside **Docker**. Docker packages a program together with everything it needs, so nothing else has to be installed on your laptop.

Windows

1. Download [Docker Desktop](#) and run the installer.
2. When the installer asks about **WSL 2**, leave the option switched on. Docker needs it on Windows.
3. Restart the laptop if the installer asks you to.
4. Open Docker Desktop and wait until the status in the bottom left says **Engine running**.

macOS

1. Download [Docker Desktop](#). Pick the version for your chip: **Apple Silicon** for M1 and later, **Intel** for older Macs. You can check under → *About This Mac*.
2. Open the downloaded file and drag Docker into *Applications*.
3. Open Docker Desktop and wait until the status in the bottom left says **Engine running**.

Linux

You can use either Docker Desktop or Docker Engine.

- **Docker Desktop:** follow the [instructions for your distribution](#).
- **Docker Engine:** follow the [instructions for your distribution](#), including the Compose plugin. Then allow your user to run Docker without `sudo`, and log out and back in afterwards:

```
sudo usermod -aG docker $USER
```

Check that Docker works:

```
docker compose version
```

3. Download the hackathon files

Without Git (recommended if you don't know Git):

1. Open the [project page on GitHub](#).
2. Click the green **Code** button, then **Download ZIP**.
3. Unzip the file, for example onto your desktop.

With Git:

```
git clone https://github.com/aihpi/demo-bibliothekshackaton.git
cd demo-bibliothekshackaton
```

4. Enter your settings

All settings live in a file called `.env` in the project folder. The folder contains a template, `.env.example`.

You don't need to create `.env` yourself. The first time you start Langflow (step 5), the start script copies the template to `.env`, opens it in a text editor and stops. Fill in the values, save the file and start again.

If you'd rather do it by hand, copy `.env.example`, name the copy `.env` and fill it in.

Setting	What to enter
KI_STANDARD	Which model the example flows use by default: <code>cluster</code> or <code>local</code> (local). Leave it at <code>cluster</code> unless the organisers say otherwise.
CLUSTER_BASE_URL	Address of the AI cluster, from the organisers
CLUSTER_API_KEY	Your group's key, from the organisers. Please don't share it.
CLUSTER_MODELL	Name of the model on the cluster, from the organisers
LOKAL_BASE_URL	Address of the local model. Only change it if you use Ollama in Docker (see step 7).
LOKAL_MODELL	Name of the local model, <code>qwen3.5:4b</code> by default
LOKAL_KONTEXT	How much text the local model reads at once. Leave it as it is.
KONTAKT_EMAIL	An email address. Crossref then answers faster, and Unpaywall finds more open full texts.
OPENALEX_API_KEY	A free key from openalex.org . Without one, OpenAlex slows down searches when many people use it, and all groups on the same Wi-Fi share the limit.

Hidden file on macOS: files whose names start with a dot are hidden in the Finder. Press `⌘ + ⬆ + .` (Command, Shift, full stop) to show them.

5. Start Langflow

Make sure Docker Desktop is running (or, on Linux, the Docker service). Then:

- **Windows:** double-click `starten.bat`.
- **macOS:** double-click `starten.command`. If macOS refuses to open it, see Troubleshooting.
- **Linux:** in a terminal in the project folder, run `./starten.command`.

The first start downloads Langflow, which takes a few minutes. The script waits until Langflow is ready and then opens <http://localhost:7860> in your browser.

Instead of the start scripts you can also use a terminal in the project folder:

```
docker compose up -d
```

Then open <http://localhost:7860> yourself once Langflow is ready, usually after a minute.

6. Check that everything works

1. In Langflow, open the project **Starter Project**. The example flows are there.

2. Open **00 Erste Schritte – Hallo KI** (First steps: hello AI).
3. Click **Playground** in the top right, type a question such as *Was ist eine DOI?* ("What is a DOI?") and press Enter.

If an answer appears after a few seconds, everything is set up. If you get an error instead, see [Troubleshooting](#).

The guide for participants (German) continues from here.

7. Optional: a local AI model

By default the flows use the AI cluster. A local model runs entirely on your laptop, so no data leaves it. That matters for unpublished theses, for example. Local models are smaller, slower and make more mistakes, though.

The local model is provided by [Ollama](#). There are three ways to run it:

Option	When to use it	LOKAL_BASE_URL in .env
Ollama app on the laptop	macOS (uses the graphics unit), Windows with an NVIDIA graphics card	<code>http://host.docker.internal:11434/v1</code> (the default)
Ollama in Docker	Linux, or if you don't want to install anything else	<code>http://ollama:11434/v1</code>
Ollama on one computer in the room , shared by everyone	a few powerful computers and many weaker laptops	<code>http://<ip-address>:11434/v1</code>

Install the Ollama app

- **Windows:** download `OllamaSetup.exe` from ollama.com/download and run it. Ollama then runs in the background, with an icon in the taskbar notification area.
- **macOS:** download the app from ollama.com/download, open the downloaded file and drag Ollama into *Applications*. Start it once. It then runs in the background, with an icon in the menu bar.
- **Linux:** run this in a terminal. It installs Ollama and sets it up as a service that starts automatically:

```
curl -fsSL https://ollama.com/install.sh | sh
```

Check that Ollama is running: open <http://localhost:11434> in your browser. It should say **Ollama is running**. You can also check in a terminal:

```
ollama --version
```

Linux with Docker Engine: Ollama only listens on `127.0.0.1` by default, and Langflow in Docker can't reach that address. Either use Ollama in Docker, or let Ollama listen on all addresses: run `sudo systemctl edit ollama`, add the two lines below, then run `sudo systemctl restart ollama`. Docker Desktop doesn't need this.

```
[Service]
Environment="OLLAMA_HOST=0.0.0.0"
```

Download the first model

Open a terminal (on Windows: *PowerShell* or *Command Prompt*) and run:

```
ollama pull qwen3.5:4b
```

This downloads about 3.4 GB and takes a few minutes, depending on your connection. When it's finished, check that the model is there:

```
ollama list
```

Then ask it a first question. The first answer can take a little longer, because the model has to be loaded into memory:

```
ollama run qwen3.5:4b "Was ist eine DOI?"
```

If an answer appears, the model works. The laptop can now answer without the internet, although the library components still need it to reach the databases.

Ollama in Docker instead of the app

```
docker compose --profile lokal up -d
```

This starts Ollama in Docker and downloads the model named in `LOKAL_MODELL` the first time (a few GB). Set `LOKAL_BASE_URL=http://ollama:11434/v1` in `.env`. On a Mac this option doesn't use the graphics unit and is much slower than the app.

To download another model into the Docker version of Ollama:

```
docker compose exec ollama ollama pull qwen3.5:9b
```

Ollama on a shared computer

Start Ollama there with the environment variable `OLLAMA_HOST=0.0.0.0` so other laptops can reach it, and enter that computer's IP address in `LOKAL_BASE_URL`.

Which model?

- `qwen3.5:4b` (3.4 GB) runs on almost any laptop.
- `qwen3.5:9b` (6.6 GB) gives clearly better results but needs 16 GB of RAM.

If you use a model other than `qwen3.5:4b`, enter its name in `LOKAL_MODELL`.

Use the local model in Langflow

- **For all flows:** set `KI_STANDARD=lokal` in `.env`, then run `docker compose up -d` again so Langflow picks up the new setting.
- **For a single flow:** in the *KI-Modell* block, set **Quelle** (source) to *Lokal (Ollama)*.

8. For organisers: preparing many laptops

Prepare each laptop with steps 2 to 5: install Docker, copy the project folder, put the group's `.env` into it, and start Langflow once so the download is done before the event.

Save the Wi-Fi: if many laptops download Langflow at the same time, the network gets slow. Download the image once and copy it to the laptops on a USB stick:

```
docker pull langflowai/langflow:1.12.3
docker save langflowai/langflow:1.12.3 -o langflow-1.12.3.tar

# then on each laptop:
docker load -i langflow-1.12.3.tar
```

The guide for organisers (German) covers credentials, a test run and the schedule for the day.

9. Stop, update and remove

Stop

In Docker Desktop, stop the container whose name starts with `demo-bibliothekshackaton`, or run this in the project folder:

```
docker compose down
```

Your own flows are kept.

Update to the latest version

Before you update:

- **Export the flows you built yourself**, to be safe: in Langflow, open the flow's menu and choose **Export**.
- **Don't change the example flows directly**. Every time Langflow starts, it replaces the example flows in *Starter Project* with the versions from the project folder. Changes made directly in an example flow are lost, with or without an update. Work on a copy instead: in the overview, open the flow's menu and choose **Duplicate**.

With Git: run this in a terminal in the project folder:

```
docker compose down
git pull
docker compose up -d
```

Stopping Langflow first makes sure it starts fresh and loads the new components and example flows. `git pull` doesn't touch your `.env`.

If `git pull` refuses because you changed a file of the project yourself (for example the port in `docker-compose.yml`), put your changes aside, update, and then bring them back:

```
git stash
git pull
git stash pop
```

Without Git:

1. Stop Langflow (see above).
2. Download and unzip the new version (step 3). It doesn't matter what the new folder is called.
3. Copy `.env` from the old folder into the new one.
4. Start Langflow from the new folder (step 5). Your own flows are still there. Once everything works, you can delete the old folder.

Remove everything

```
docker compose down -v
docker image rm langflowai/langflow:1.12.3
```

Careful: `down -v` deletes all flows you built yourself. Export them first.

Then delete the project folder, and uninstall Docker Desktop if you no longer need it.

10. Troubleshooting

Problem	Solution
The start script says Docker isn't running	Open Docker Desktop and wait for Engine running , then start again.
Docker Desktop doesn't start on Windows	WSL 2 is missing. Open a command prompt as administrator, run <code>wsl --install</code> and restart. If that doesn't help, virtualisation may be switched off in the BIOS.
macOS won't open <code>starten.command</code>	Right-click the file and choose Open . If that doesn't work, go to <i>System Settings</i> → <i>Privacy & Security</i> and click Open Anyway .
"Permission denied" when starting on macOS or Linux	In a terminal in the project folder, run <code>chmod +x starten.command</code> .
Linux: "permission denied" when Docker connects	Your user isn't in the <code>docker</code> group yet (see step 2). Log out and back in after adding it.
The browser says the page can't be reached	Langflow is still starting. Wait a minute and reload the page.
Port 7860 is already in use	In <code>docker-compose.yml</code> , change " <code>7860:7860</code> " to e.g. " <code>7861:7860</code> " and open http://localhost:7861 .
The download stops or is very slow	A company proxy or busy Wi-Fi. Load the image from a USB stick (step 8).
"401" or "Authentication" in the flow	<code>CLUSTER_API_KEY</code> is wrong. Ask the organisers.
"Kein Modell gewählt" (no model selected)	Set <code>CLUSTER_MODEL</code> in <code>.env</code> , or pick a model in the <i>KI-Modell</i> block.
Local model: "connection refused"	Ollama isn't running or can't be reached. See step 7, including the note for Linux.
Changes to <code>.env</code> have no effect	Run <code>docker compose up -d</code> again.

Problem	Solution
Anything else	Look at Langflow's log with <code>docker compose logs -f langflow</code> , or ask the organisers.

More problems you might run into while working with the flows are listed in the guide for participants (German).